Comprehensive Analysis of Firmware Integrity, ACPI Namespace Collisions, and Secure Boot Kernel Module Restrictions on Modern Platforms

Architectural Overview and Systemic Context

The modern computational platform is defined by a highly orchestrated, deeply interdependent relationship between hardware, low-level firmware, and the operating system kernel. The Unified Extensible Firmware Interface (UEFI) and the Advanced Configuration and Power Interface (ACPI) act as the foundational bridges that translate physical electrical topologies into logical, manageable software constructs. When an operating system initializes, it relies entirely on the structural integrity, syntactic correctness, and cryptographic authenticity of these firmware interfaces to construct its internal representation of the machine.

Diagnostic artifacts extracted from a Lenovo ThinkSystem/ThinkPad environment, operating under a Linux kernel (version 6.12.0-124.40.1.el10_1.x86_64), reveal profound systemic friction occurring at multiple layers of this initialization stack.[1] The empirical data exposes two primary categories of operational anomalies. First, the ACPI Component Architecture (ACPICA) interpreter encounters persistent AE_ALREADY_EXISTS exceptions during the dynamic namespace evaluation phase, indicating fundamental table-load collisions and runtime idempotency failures within the compiled ACPI Machine Language (AML) bytecodes.[1] Second, the operating system enforces an uncompromising cryptographic perimeter, leveraging EFI Secure Boot, Kernel Lockdown modules, and Federal Information Processing Standards (FIPS) to actively reject the insertion of out-of-tree hardware assessment drivers, specifically the CHIPSEC kernel module.[1]

This report provides an exhaustive, granular examination of these phenomena. It dissects the mechanical execution of the ACPI namespace parser, categorizes the precise nature of the Universal Serial Bus (USB) topology collisions, evaluates the lexical failure of the ACPI source language disassembly toolchain, and scrutinizes the cryptographic mechanisms governing kernel module rejection.

The ACPI Namespace and the Mechanics of AE_ALREADY_EXISTS

The ACPI namespace is not a static data structure; it is a dynamically instantiated, hierarchical tree that the operating system constructs during the early phases of the boot sequence.[1] The

Linux kernel incorporates the ACPICA reference implementation to ingest the primary Differentiated System Description Table (DSDT) and subsequently load multiple Secondary System Description Tables (SSDTs).[1] These tables contain AML, a bytecode language that is parsed and executed by the OS to map hardware pathways, define power management profiles, and establish thermal control zones.[1]

During this ingestion process, the ACPICA interpreter catalogs every named object—such as devices, methods, thermal zones, and buffer fields—into the namespace dictionary.[1] When the interpreter encounters a definition block that attempts to declare a named entity that already possesses an active memory allocation within the logical hierarchy, it immediately triggers an AE_ALREADY_EXISTS exception.[1] The kernel logs prominently feature these errors occurring within the dswload2-326 and psobject-220 execution paths of the ACPICA subsystem.[1] The dswload2 phase represents the second pass of the dynamic AML loading mechanism, where unresolved operands are evaluated and object creation opcodes are finalized.

Class A Collisions: Static Table-Load Duplication

The highest frequency of namespace collisions documented in the diagnostic output falls into the Class A category, characterized by static table-load duplication.[1] This architectural flaw typically arises when modern, modular UEFI firmware compilation processes merge discrete SSDT payloads that inadvertently overlap with objects already defined in the baseline DSDT.[1]

Global Object Conflicts: \GPLD and \GUPC

The most fundamental manifestation of this duplication occurs at the absolute root of the namespace, specifically with the \GPLD and \GUPC control methods.[1] Terminal outputs detailing a recursive search across the disassembled ACPI Source Language (DSL) files expose the widespread, redundant invocation of these methods.[1]

ASL Definition	Invocation Syntax	Argument 1	Argument 2
Method (GPLD, 2, Serialized)	Return (GPLD (One, One))	Connectivity Boolean	Port Group/Token
Method (GUPC, 2, Serialized)	Return (GUPC (One, Zero))	Visibility Boolean	Connector Type

Method (GPLD, 2, Serialized)	Return (GPLD (Zero, Zero))	Connectivity Boolean	Port Group/Token
Method (GUPC, 2, Serialized)	Return (GUPC (Zero, 0xFF))	Visibility Boolean	Connector Type

In standard firmware architecture, GPLD (Get Physical Location of Device) and GUPC (Get USB Port Capabilities) serve as global helper methods.[1] Rather than embedding complex, multi-byte static buffers into every individual USB port definition, firmware engineers utilize these serialized methods to return standardized packages based on simple input arguments.[1] For instance, a call to GUPC (One, Zero) dictates that a port is connectable (One) and utilizes a standard Type-A connector (Zero). Conversely, GUPC (Zero, 0xFF) dictates that the port is hardwired or non-user-visible (Zero) and utilizes a proprietary or internal connector (0xFF).[1]

Because these methods are declared as Serialized, the ACPICA interpreter creates a localized mutex to prevent reentrancy issues during execution. However, the root cause of the AE_ALREADY_EXISTS error is not execution concurrency, but rather the structural re-declaration of the Method (GPLD,...) block itself across multiple loaded tables.[1] When an SSDT attempts to define \GPLD after it has already been established by the DSDT, the namespace strictly rejects the re-declaration, resulting in the logged exception.[1]

USB Subsystem Topology Modeling

The architectural impact of these Class A collisions is most visible within the eXtensible Host Controller Interface (XHCI) routing logic.[1] Modern Intel platforms utilize XHCI to manage all Universal Serial Bus traffic, logically separating physical connectors into High Speed (HS) ports for USB 2.0 backward compatibility and SuperSpeed (SS) ports for high-bandwidth USB 3.x signaling.

The system logs exhibit a cascading failure sequence across the entire _SB.PC00.XHCI.RHUB path, encompassing ports HS01 through HS14 and SS01 through SS10.[1] For nearly every enumerated port, the firmware attempts and fails to instantiate two critical sub-objects:

1. **_UPC (USB Port Capabilities):** An ACPI object that evaluates to a package detailing the electrical and mechanical routing configuration of the port. The OS relies on this package to inform user-space daemons, such as usbguard, about whether a port is exposed to the external chassis or permanently wired to an internal component (e.g., an integrated biometric sensor or Bluetooth transceiver).[1]
2. **_PLD (Physical Location of Device):** An ACPI object that evaluates to a highly structured, 128-bit (minimum) buffer. This buffer defines the exact physical geometry of the port on

the system chassis, encoding variables such as vertical offset, horizontal offset, panel surface (front, back, top, bottom), shape, and color.[1]

The continuous AE_ALREADY_EXISTS exceptions for these specific objects indicate that the UEFI firmware is pushing conflicting topology maps.[1] Industry documentation and vendor advisories for systems like the Lenovo ThinkSystem SR950 confirm that such errors occur when USB ports have the _UPC and _PLD methods defined in multiple locations, such as overlapping SSDTs generated during different stages of the OEM compilation process.[3]

While the ACPICA interpreter is engineered to handle these exceptions gracefully—typically by ignoring the subsequent re-declarations and operating on the initial definition—this conflict introduces a risk of topological metadata degradation.[1] If an overlapping SSDT was intentionally designed to act as a dynamic override (for instance, dynamically altering the _PLD buffer of HS05 to reflect an externally accessible port when a specific docking station is detected), the OS's refusal to process the duplicate object means the system will operate on baseline, potentially inaccurate chassis data.[1] This metadata corruption directly influences peripheral power management and hardware access policies.

Class B Collisions: Runtime Idempotency Failures

Distinct from static table-load duplication, the diagnostic logs reveal a sophisticated runtime conflict at the device path _SB.PC00.PEG1.PEGP._DSM.USRG.[1] The kernel logs the error as CreateBufferField failure (20240827/dswload2-477) and subsequently aborts the execution of the _DSM method entirely due to an AE_ALREADY_EXISTS exception.[1]

The _DSM (Device Specific Method) is an advanced ACPI mechanism that empowers the operating system to invoke proprietary, hardware-specific control functions that transcend standardized ACPI definitions.[1] At the PEG1.PEGP path, this method interacts with the PCI Express Graphics Port, governing the behavior of a discrete graphics processing unit (GPU).[1] The Direct Rendering Manager (DRM) and PCIe subsystems in the Linux kernel repeatedly invoke this _DSM to negotiate power state transitions, initialize display multiplexers, and validate link active states.[1]

The failure mechanism here is an absence of execution idempotency.[1] Within the AML instructions of the GPU's _DSM, the firmware developer utilized an opcode—such as CreateBufferField, CreateWordField, or CreateDWordField—to dynamically instantiate the variable USRG.[1] This field is likely used to temporarily store execution states or parse incoming arguments. However, because a _DSM is intended to be called iteratively throughout the system's operational lifecycle, its internal code must be structurally idempotent.

If the AML lacks a logical guard constraint—such as executing an If (LNot (CondRefOf (USRG))) check prior to field creation—the ACPICA interpreter will attempt to create the USRG object in the namespace every single time the _DSM is invoked.[1] During the initial boot phase, the first

invocation succeeds. On the second invocation, the interpreter detects that USRG is already globally registered, triggers the AE_ALREADY_EXISTS exception, and fatally aborts the method.[1]

This unhandled termination deprives the kernel's PCIe driver of critical return values. Without the data expected from the _DSM, the operating system may fail to place the discrete GPU into its deepest low-power states (such as D3cold), leading to elevated thermal output, degraded battery longevity, and sub-optimal peripheral management.

Lexical Parsing Failures and Firmware Auditing Toolchains

To definitively diagnose and trace the origins of these namespace collisions, an analyst must extract the compiled binary tables from system memory and reverse-engineer them into human-readable ACPI Source Language (DSL). This requires the rigorous application of standard firmware auditing toolchains, specifically acpidump, acpixtract, and the Intel ASL compiler/disassembler iasl.[1]

The empirical data demonstrates the successful execution of acpidump followed by acpixtract -a, which isolated an extensive array of binary definition blocks from the system's physical memory.[1] The output illustrates the sheer volume and complexity of the firmware payload on the target platform:

Table Signature	Extracted Byte Length	Architectural Function
DSDT	598,892	Differentiated System Description Table; the primary namespace anchor.
LUFT	216,258	OEM-specific proprietary definition table.
SSDT 4	37,620	Secondary System Description Table; one of 17 modular definition blocks.

FACP	276	Fixed ACPI Description Table; dictates baseline hardware registers and power models.
APIC	476	Multiple APIC Description Table; governs symmetric multiprocessor interrupt routing.
TPM2	76	Trusted Platform Module 2.0 specification block.
DMAR	80	DMA Remapping Table; critical for IOMMU and VT-d enforcement.
WSMT	40	Windows SMM Security Mitigations Table.

The "Unterminated Quoted String" Collapse

Following the successful extraction of the binary tables, the application of iasl -d *.dat resulted in a catastrophic compilation failure.[1] The disassembler output flooded the terminal with over 200 sequential errors stating Unterminated quoted string, ultimately aborting the process with a Maximum error count (200) exceeded fatal stop.[1]

This massive lexical collapse exposes a critical vulnerability in the ASL parsing engine when confronted with modern, blob-heavy UEFI environments. When the iasl disassembler reads a raw binary .dat file, it maps hexadecimal values against standard AML opcodes and ASCII representations.[1] The Lenovo firmware contains numerous non-standard, OEM-specific binary tables, such as the massive 216 KB LUFT table and embedded Microsoft Data Management (MSDM) arrays.[1]

If the parser encounters a raw byte with the hexadecimal value of 0x22 inside a binary blob or an embedded certificate, it incorrectly interprets this byte as the initiation of an ASCII string

literal (an opening quotation mark). Because the surrounding data is compiled binary rather than text, a corresponding closing 0x22 byte may not appear before the parser reaches an illegal control character, a carriage return, or the end of the file.[1] This structural misinterpretation causes the lexical scanner to consume vast tracts of the binary file as an invalid string, generating cascading line errors until the compiler's internal error threshold is violently breached.[1]

Furthermore, the specific error Invalid field label detected (found "0000" expected "Signature") reported against acpi_dump.dat demonstrates the importance of rigorous file filtering.[1] The user attempted to disassemble the raw, concatenated output of acpidump directly, rather than limiting the operation to the cleanly segregated outputs of acpixtract. The ASL compiler expects strict table headers (e.g., DSDT, SSDT); feeding it a concatenated text dump destroys its ability to align memory offsets, resulting in immediate structural rejection.[1]

Resolution of External Control Methods

A secondary challenge in ACPI namespace auditing involves the dependency graphs between discrete tables. The disassembled output of the DSDT [1] explicitly notes that the ASL compiler encountered external control methods (e.g., _PR_.CPU0._PPC, CFGD, WMAB) that it could not resolve.[1] In an isolated disassembly, iasl tags these as UnknownObj because it cannot infer the required argument counts from a missing SSDT payload.[1]

To generate a mathematically and syntactically verifiable disassembly that can definitively prove the source of an AE_ALREADY_EXISTS collision, the execution workflow must load all dependencies simultaneously.[1] The optimal operational command requires the use of the external reference flag: iasl -e ssdt*.dat -d dsdt.dat.[1] This syntax forces the compiler to build a holistic, multi-table namespace in memory before generating the DSL output. By pre-loading the SSDTs, the compiler can accurately map the argument counts and return types of methods like GUPC across the entire hierarchy, providing the analyst with clear, uncorrupted evidence of redundant object declarations.[1]

Cryptographic Kernel Perimeters and Module Rejection

Parallel to the firmware namespace investigation, an attempt was made to deploy the CHIPSEC framework.[1] CHIPSEC is a highly specialized platform security assessment framework utilized by researchers to evaluate hardware interfaces, SPI flash write protections, System Management Mode (SMM) vulnerabilities, and Secure Boot configurations. To perform direct reads and writes to physical memory, PCI configuration spaces, and Model-Specific Registers (MSRs) from user-space, the framework relies on a custom, out-of-tree kernel driver named chipsec.ko.[1]

The terminal sequence illustrates a systematic failure during the deployment phase. After cloning the repository and executing python3 setup.py build to compile the C-extensions and the Linux driver, the user attempted to escalate privileges and inject the driver into the running kernel using sudo insmod. The kernel immediately blocked the operation, returning the fatal error: insmod: ERROR: could not insert module... Key was rejected by service.[1]

This rejection is not indicative of a software defect, a compilation error, or a missing dependency; it is the deliberate, mathematically enforced behavior of the Linux kernel operating under the EFI Secure Boot paradigm.[1]

The Mechanics of Linux Kernel Lockdown

When the physical hardware initializes under UEFI Secure Boot, the system bootloader (e.g., GRUB) verifies the cryptographic signature of the Linux kernel against the public certificates stored in the non-volatile UEFI Signature Database (db).[1] Upon a successful mathematical validation, the kernel establishes a chain of trust. To preserve the integrity of this trust boundary throughout the system's uptime, the kernel automatically enables the Linux Kernel Lockdown Security Module (LSM).[1]

The diagnostic logs are saturated with continuous notifications of these restrictions being enforced: Lockdown: systemd-logind: hibernation is restricted; see man kernel_lockdown.7 and Lockdown: systemd: /dev/mem,kmem,port is restricted.[1] The Lockdown LSM operates under two primary thresholds: *integrity* and *confidentiality*.

In *integrity* mode, the kernel strictly prohibits any user-space process—even those running with CAP_SYS_ADMIN (root) privileges—from modifying the running kernel code or manipulating physical hardware states.[1] This is achieved by blocking direct access to /dev/mem, disabling the use of the ioperm and iopl system calls, and restricting the use of custom BPF (Berkeley Packet Filter) programs.[1] In *confidentiality* mode, the kernel extends these protections to prevent user-space from extracting sensitive cryptographic material from kernel memory, disabling features like unencrypted hibernation and restrictive kcore access.[1]

Mandatory Module Signing and PKCS#7 Verification

Because the kernel guarantees its own execution environment under Secure Boot, it extends this mandate to all Loadable Kernel Modules (LKMs).[1] It is a logical fallacy to secure the core kernel but allow an unverified driver to execute in ring-0, where it could bypass all security controls. Therefore, the init_module and finit_module system calls are hooked by the verification subsystem.

When a module is passed to the kernel, the subsystem extracts the PKCS#7 cryptographic signature appended to the end of the .ko binary.[1] The kernel validates this signature against the keys held in the .builtin_trusted_keys keyring and the Secondary Trusted Keyring.[2]

When the user compiled the chipsec.ko module locally via the python setup script, the resulting binary was either completely unsigned or signed with an ephemeral, auto-generated build key that the running kernel did not possess in its trusted databases.[1] Consequently, when insmod triggered the module load request, the verification algorithm failed, and the cryptographic service rejected the untrusted key, resulting in the reported error.[1]

Provisioning Out-of-Tree Modules via the MOK Infrastructure

To bypass this restriction and successfully execute hardware assessment frameworks without permanently disabling UEFI Secure Boot—which would dangerously degrade the platform's defensive posture—system administrators must utilize the Machine Owner Key (MOK) facility.[6] The MOK infrastructure, typically managed by the shim bootloader and the mokutil user-space tool, allows the platform owner to securely bridge local trust into the kernel.[6]

The cryptographic lifecycle required to load the CHIPSEC driver under these constraints involves several strict operational phases:

Provisioning Phase	Cryptographic Action	Architectural Implication
Generation	Execute OpenSSL to create an X.509 certificate and an RSA private key pair.	Establishes a local cryptographic authority independent of the OEM or Microsoft.
Signing	Utilize the kernel scripts/sign-file utility to append a PKCS#7 signature to the chipsec.ko file.	Cryptographically binds the compiled module to the locally generated private key.
Staging	Execute mokutil --import <public_key.der>.	Stages the public key into a volatile EFI variable, prompting evaluation on the next boot cycle.
Enrollment	Reboot the system and interface with the	Physically authenticates the key enrollment, requiring a

	MokManager utility via the pre-boot UEFI console.	human presence to mitigate remote code injection attacks.
Execution	Execute sudo insmod chipsec.ko.	The kernel successfully validates the module's PKCS#7 signature against the newly enrolled MOK database and permits insertion.

Until this specific cryptographic chain of trust is deliberately established by the system administrator, advanced diagnostic tools requiring direct hardware access will remain permanently blockaded by the system's integrity perimeter.[1]

Systemic Physical Guardrails: Memory Maps, IOMMU, and FIPS

The extensive dmesg output provides profound insight into how the Linux kernel negotiates the physical hardware landscape defined by the firmware, highlighting strict architectural boundaries that prevent systemic overlap and memory corruption.[1]

BIOS e820 Memory Topologies and NVS Constraints

During early boot initialization, the firmware passes the e820 memory map to the kernel. This map dictates the strict categorization of physical address ranges, designating segments as usable RAM, reserved space, ACPI data, or ACPI Non-Volatile Sleep (NVS) memory.[1]

The kernel logs demonstrate rigorous adherence to these designations. For example, the kernel acknowledges the exact physical bounds of the ACPI NVS memory at [mem 0x00000000758fc000-0x0000000075a20fff].[1] This precise mapping is critical for system stability. ACPI NVS memory stores vital state information utilized by the System Management Mode (SMM) and the OS during complex power state transitions, such as entering an S3 suspend or performing an S0 wake.[1] If the kernel were to ignore the e820 map and utilize this region for standard page allocations or DMA buffers, the firmware could arbitrarily overwrite active OS memory during a sleep transition, resulting in catastrophic corruption and kernel panics.

Furthermore, the kernel proactively registers specific reserved memory blocks intended to prevent hibernation conflicts: PM: hibernation: Registered nosave memory: [mem 0x75a21000-0x767fefff].[1] This explicit physical routing ensures that despite any logical

namespace overlaps occurring in the ACPI tree (the AE_ALREADY_EXISTS errors), the underlying physical memory architecture remains uncompromised and structurally sound.

IOMMU Configuration and Driver Conflict Prevention

The operating environment is further hardened by the activation of the Intel Input-Output Memory Management Unit (IOMMU), as indicated by the intel_iommu=on iommu=pt command line parameters and the initialization of the Direct Memory Access Remapping (DMAR) tables.[1] The IOMMU governs DMA requests from peripheral devices, mapping device-visible virtual addresses to physical memory addresses. This prevents malicious or malfunctioning hardware (such as a compromised Thunderbolt device) from executing DMA attacks against sensitive kernel memory spaces. The logs indicate the successful grouping of PCI devices into isolated IOMMU trust domains, ensuring strict peripheral compartmentalization.[1]

Additionally, the kernel command line implements a rigorous driver preemption strategy: rd.driver.blacklist=nouveau modprobe.blacklist=nouveau nouveau.modeset=0.[1] The nouveau driver is the open-source implementation for NVIDIA graphics hardware. Blacklisting it at the initramfs level prevents the kernel from loading nouveau and causing a severe resource collision with the proprietary nvidia driver module over the PCI Express endpoint mapping.[1] This explicit command ensures that the Direct Rendering Manager (DRM) initializes cleanly, preventing two competing drivers from attempting to write to the same VGA arbitration registers simultaneously.[1]

Cryptographic Rigidity via FIPS 140-2 Enforcement

The environment operates under the extreme constraints of the Federal Information Processing Standard (FIPS) mode, initialized by the kernel parameter fips=1.[1] When FIPS mode is enforced, the Linux cryptographic API is mandated to execute continuous self-tests and fundamentally disable any cipher, hash, or algorithm that has not been explicitly approved by the National Institute of Standards and Technology (NIST).[1]

The execution of these constraints is highly visible in the boot logs. The kernel successfully validates FIPS-approved algorithms, generating success metrics for critical operations: alg: self-tests for ecdsa-nist-p521-generic (ecdsa-nist-p521) passed, alg: self-tests for sha512 (sha512) passed, and alg: self-tests for xts(aes) (xts(aes)) passed.[1]

Conversely, the kernel systematically disables non-compliant algorithms. The logs reflect the immediate suppression of these functions: alg: xchacha20 (xchacha20-simd) is disabled due to FIPS, alg: poly1305 (poly1305-simd) is disabled due to FIPS, and alg: des3_ede (des3_ede-generic) is disabled due to FIPS.[1] While FIPS enforcement drastically elevates the regulatory compliance and cryptographic certainty of the platform, it introduces a layer of operational rigidity. User-space utilities, VPN clients, and secure tunnel applications are forced to negotiate exclusively using the whitelisted FIPS cipher suites, preventing the use of modern,

highly efficient algorithms like ChaCha20-Poly1305.

Strategic Syntheses and Remediation Directives

Evaluating the confluence of firmware architectural flaws (ACPI namespace collisions) alongside uncompromising security perimeters (Secure Boot, Lockdown, and FIPS) requires a highly structured mitigation strategy. While the symptoms manifest in the kernel, the root causes span the entire vertical stack.[1]

Assessing the Security Ramifications of Namespace Collisions

The repeated AE_ALREADY_EXISTS errors generated by the \GPLD, \GUPC, and _UPC / _PLD collisions are generally classified as firmware correctness issues rather than critical security vulnerabilities.[1] The ACPICA interpreter is designed with defensive fault tolerance; it handles these exceptions gracefully by discarding the redundant definitions and operating based on the first iteration loaded into memory.[1]

However, in specialized enterprise environments, these collisions can subtly degrade endpoint security policies.[1] If an organization utilizes the usbguard daemon to automatically whitelist internal USB interfaces (e.g., an integrated webcam) while enforcing strict block policies on external chassis ports, the daemon relies entirely on the accuracy of the _UPC and _PLD metadata to enforce this boundary.[1] If a table-load collision causes the kernel to discard an updated SSDT that maps a specific port to an external header, replacing it with a generic DSDT definition that classifies it as internal, an attacker could potentially bypass the usbguard blocklist by plugging a malicious device into the misclassified port.[1]

Similarly, the failure of the _DSM.USRG method due to runtime idempotency issues deprives the operating system of critical hardware negotiation capabilities regarding the discrete GPU.[1] While this generally results in power inefficiencies or display multiplexing failures, an unhandled exception inside a highly active PCIe control loop could theoretically be exploited. A local attacker might attempt to force rapid state transitions, aiming to trigger a race condition or a NULL pointer dereference within the kernel driver managing the PCIe endpoint when the expected _DSM buffer fails to materialize.[1]

Vendor Coordination and PSIRT Disclosure

Because the root cause of the namespace collisions resides in the compiled binary payload of the UEFI firmware, operating system-level workarounds are intrinsically limited.[1] Custom kernel patches to dynamically ignore specific overlapping SSDTs are highly intrusive, difficult to maintain across kernel updates, and generally discouraged for production infrastructure.

The definitive remediation strategy requires structured coordination with the OEM firmware engineering teams.[1] The artifacts generated through proper ACPI extraction—specifically the

output of acpixtract and the cleanly resolved disassembly of iasl -e ssdt*.dat -d dsdt.dat—must be compiled into a reproducible evidence package.[1]

Prior to submission or public disclosure, this diagnostic package must undergo strict sanitization.[1] The MSDM (Microsoft Data Management) table must be securely purged, as it frequently contains the embedded Windows OEM product key in plaintext.[1] Furthermore, the kernel logs must be scrubbed of unique identifiers, including hostnames, serial numbers, and the UUIDs corresponding to LUKS-encrypted partitions.[1]

This sanitized evidence bundle should be formally submitted to the Lenovo Product Security Incident Response Team (PSIRT) under coordinated vulnerability disclosure norms.[1] Lenovo PSIRT possesses the capability to trace the redundant _UPC and _PLD definitions back to the source modules in their UEFI build environment, allowing them to release a corrected BIOS/UEFI capsule update that fundamentally resolves the AE_ALREADY_EXISTS exceptions.[1]

In conclusion, maintaining platform integrity requires a dual competency. It demands the ability to reverse-engineer and audit complex firmware hierarchies while simultaneously navigating the rigorous cryptographic constraints imposed by modern kernel security modules. The interactions documented herein serve as a quintessential study in the architectural friction between dynamic firmware models and zero-trust operating environments.

Works cited

1. ACPI Namespace Collision Finding on a Lenovo Platform.pdf
2. DKMS kernel modules give "Key was rejected by service" on Ubuntu 22.04 with 5.19.2 #906, accessed March 8, 2026, https://github.com/linux-surface/linux-surface/issues/906
3. Users may see an ACPI Error message when examining linux kernel logs - Lenovo ThinkSystem SR950, accessed March 8, 2026, https://support.lenovo.com/kr/ko/solutions/ht506123-users-may-see-an-acpi-error-message-when-examining-linux-kernel-logs-lenovo-thinksystem-sr950
4. Users may see an ACPI Error message when examining linux kernel logs - Lenovo ThinkSystem SR950, accessed March 8, 2026, https://support.lenovo.com/us/en/solutions/ht506123-users-may-see-an-acpi-error-message-when-examining-linux-kernel-logs-lenovo-thinksystem-sr950
5. We can't use Startech PEX2S953 serial card with Rocky Linux 9.2, accessed March 8, 2026, https://forums.rockylinux.org/t/we-cant-use-startech-pex2s953-serial-card-with-rocky-linux-9-2/10902
6. Kernel module error: Key was rejected by service - Hardware - openSUSE Forums, accessed March 8, 2026, https://forums.opensuse.org/t/kernel-module-error-key-was-rejected-by-service/170951
7. Users may see an ACPI Error message when examining linux kernel logs - Lenovo

ThinkSystem SR950, accessed March 8, 2026, https://support.lenovo.com/nz/en/solutions/ht506123-users-may-see-an-acpi-error-message-when-examining-linux-kernel-logs-lenovo-thinksystem-sr950